3-rdt-shap-explainer

February 23, 2024

[ ]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import shap

from catboost import CatBoostClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix
```

[ ]:
```
# set seaborn style
sns.set_style("white")
sns.set_context("talk")
```

[ ]:
```
# set random seed
seed = 42
```

[ ]:
```
# intialize shap
shap.initjs()
```

```
<IPython.core.display.HTML object>
```

[ ]:
```
# import data
df = pd.read_csv("../data/processed/corpus_studio_processed.csv")
df.drop(
    columns=["performance_type", "group"], inplace=True
)  # for this study, only studio recordings were included
df.fillna(
    "", inplace=True
)  # replace NaN values for previous and next words with empty strings
df.head()
```

[ ]:
```
       word previous_word next_word      artist song_type  \
0  hollered     conductor            cannedheat     cover
1     found                          jeffhealey     cover
2        my                      old jeffhealey     cover
3        my                          jeffhealey     cover
4     there                       is albertking     cover
```

```
                          song            aae_feature  aae_realization   time  \
0            bring_it_on_home     post-consonantal d                0  1960s
1             blue_jean_blues     post-consonantal d                1  1980s
2             blue_jean_blues  ai monophthongization                1  1980s
3             blue_jean_blues  ai monophthongization                1  1980s
4  as_the_years_go_passing_by         post-vocalic r                0  1960s

  social_group
0     nonAA_US
1  nonAA_nonUS
2  nonAA_nonUS
3  nonAA_nonUS
4           AA
```

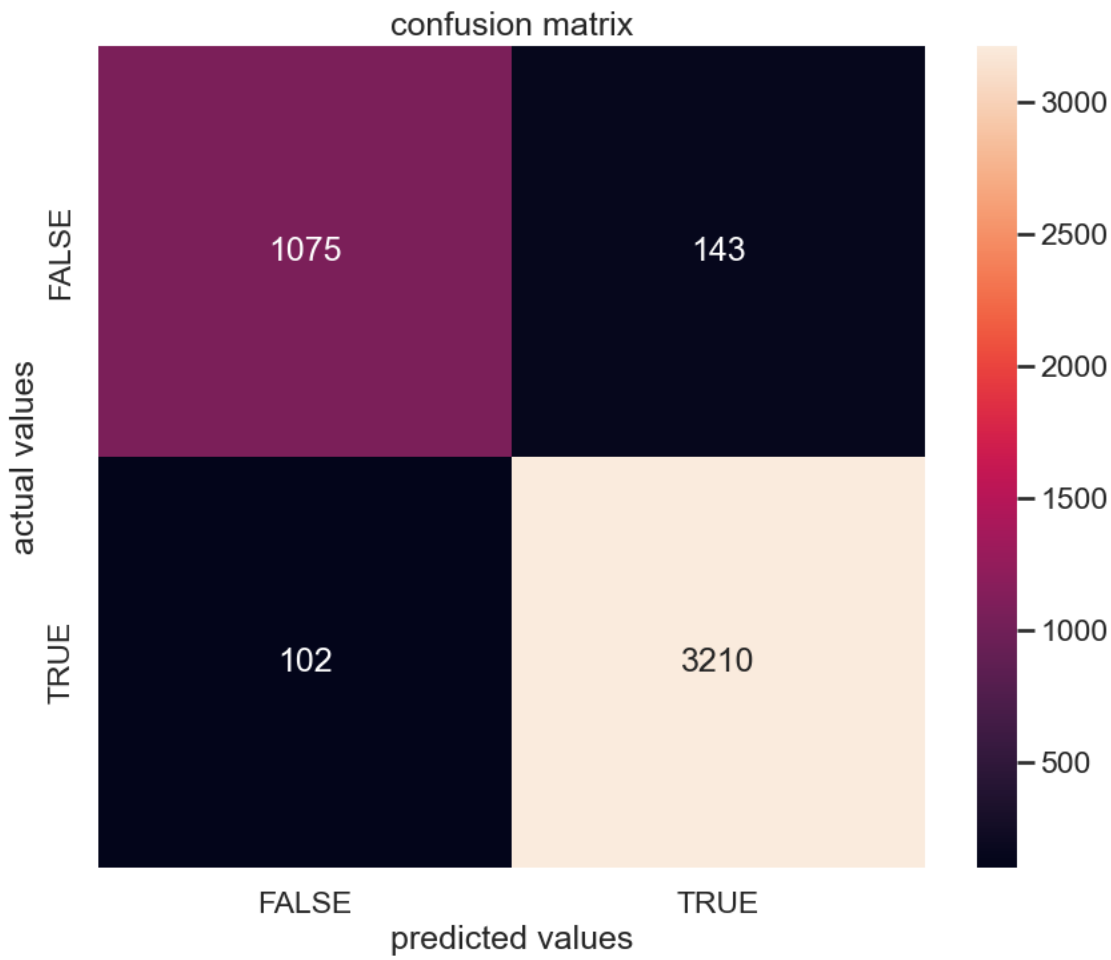
[ ]:
```
# import model
model = CatBoostClassifier()
model.load_model(fname="../models/model.json", format="json")

# define target and predictor columns
target_col = "aae_realization"
X = df.loc[:, df.columns != target_col]
y = df.loc[:, target_col]
```

[ ]:
```
# create train test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=seed
)
# get predictions
y_pred = model.predict(X_test)

# get classification report
print(classification_report(y_true=y_test, y_pred=y_pred))
```

```
              precision    recall  f1-score   support

           0       0.91      0.88      0.90      1218
           1       0.96      0.97      0.96      3312

    accuracy                           0.95      4530
   macro avg       0.94      0.93      0.93      4530
weighted avg       0.95      0.95      0.95      4530
```

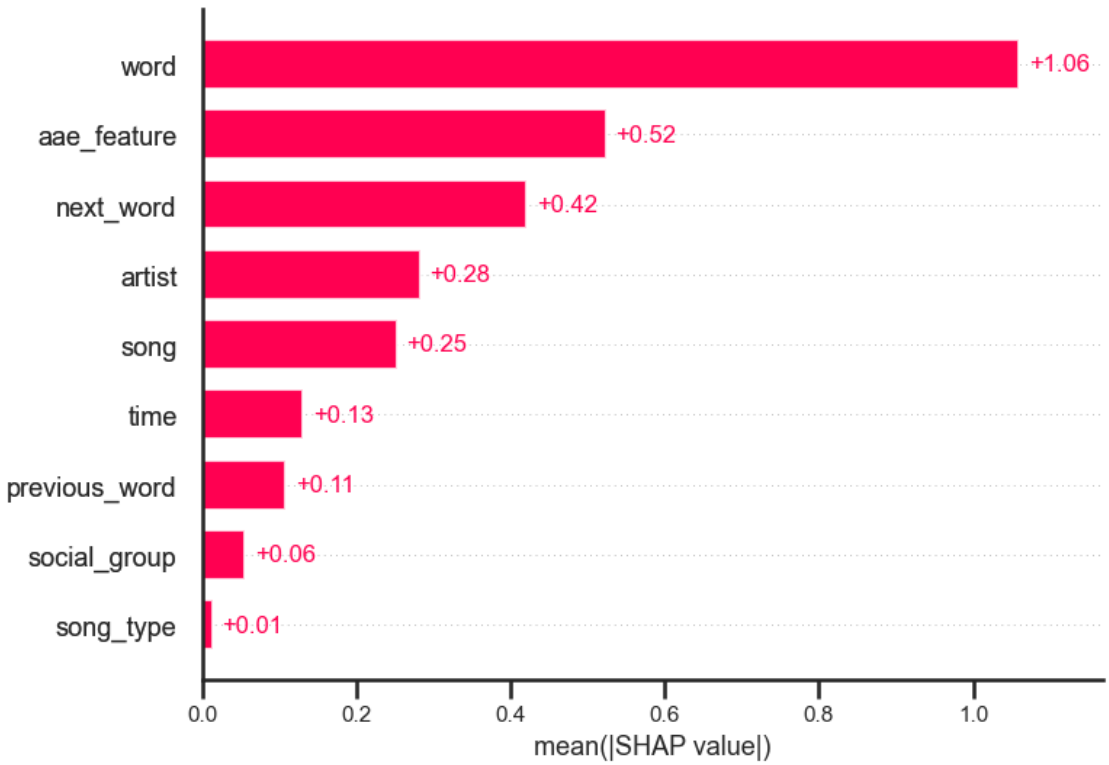
[ ]:
```
# generate confusion matrix with final model
cm = confusion_matrix(y_test, y_pred)
cm_df = pd.DataFrame(cm, index=["FALSE", "TRUE"], columns=["FALSE", "TRUE"])
plt.figure(figsize=(10, 8))
```

```
sns.heatmap(cm_df, annot=True, fmt="g")
plt.title("confusion matrix")
plt.ylabel("actual values")
plt.xlabel("predicted values")
plt.savefig("../reports/figures/confusion_matrix.png", dpi=600)
plt.show()
```

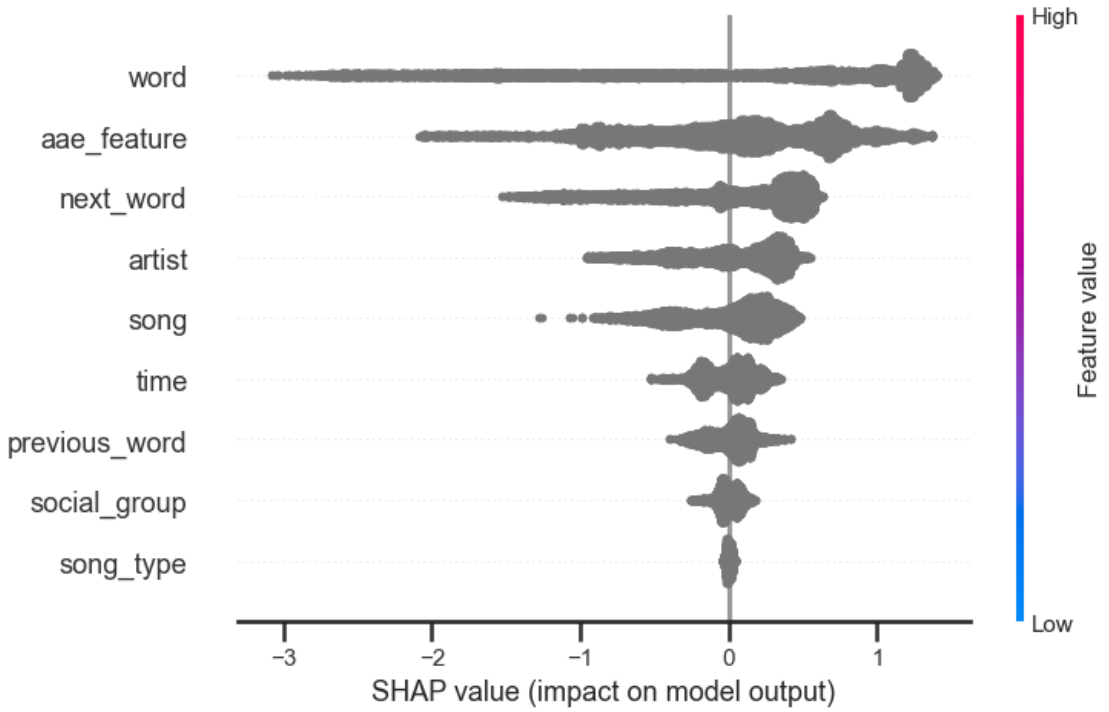


[ ]:
```
# calculate shap values
explainer = shap.TreeExplainer(model)
shap_values = explainer(X)
```

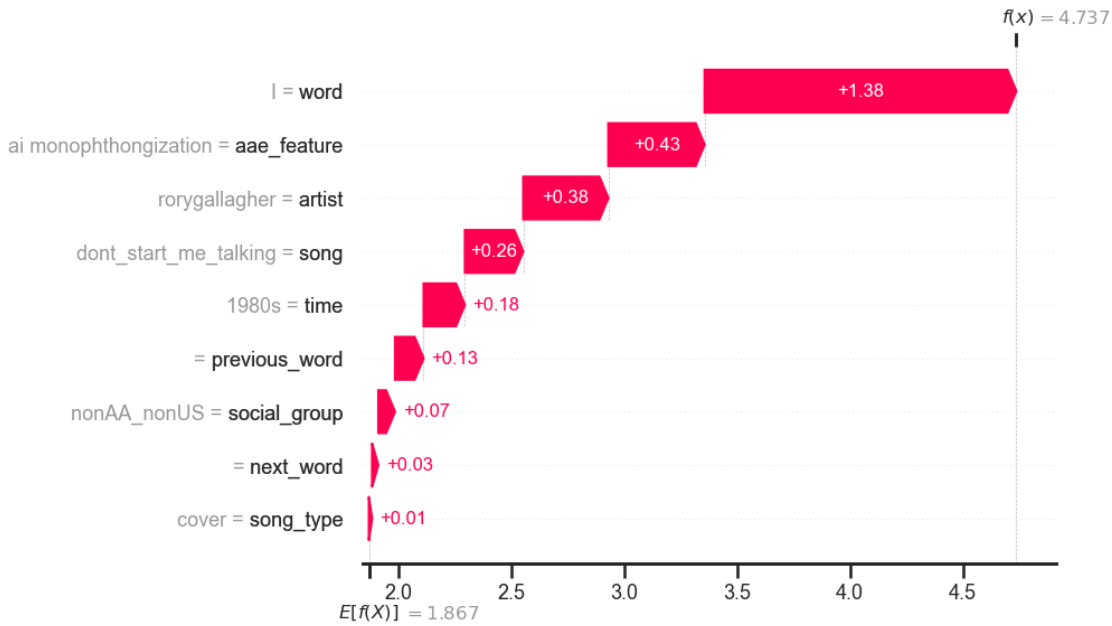
[ ]:
```
# mean shap values for each feature
shap.plots.bar(shap_values)
```



[ ]:
```
shap.summary_plot(shap_values)
```

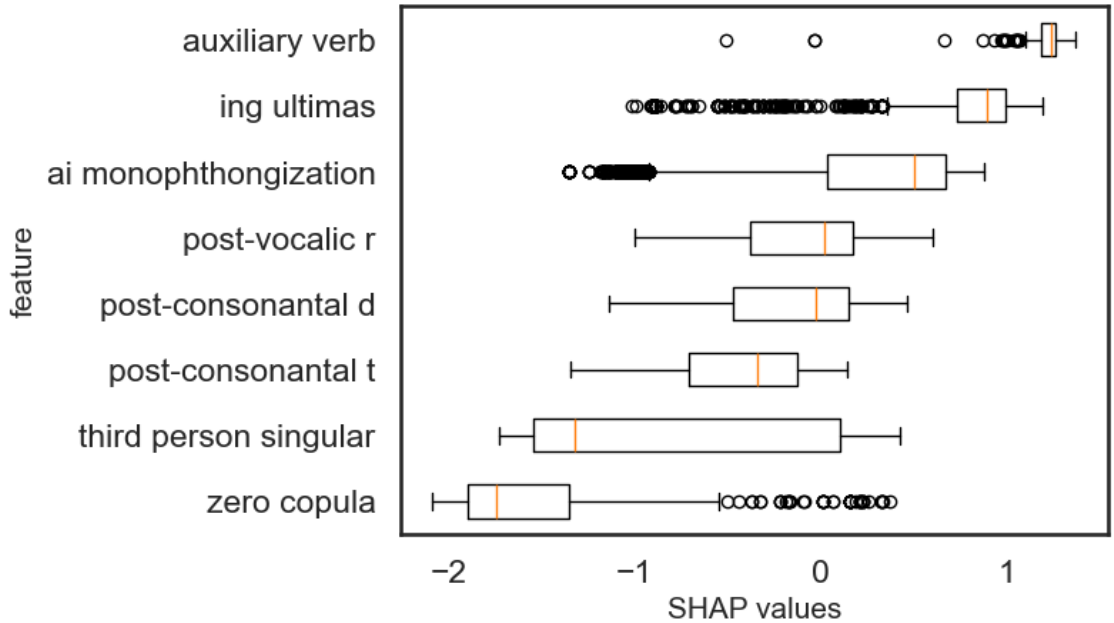


[ ]:
```
shap.plots.waterfall(shap_values[724])
```



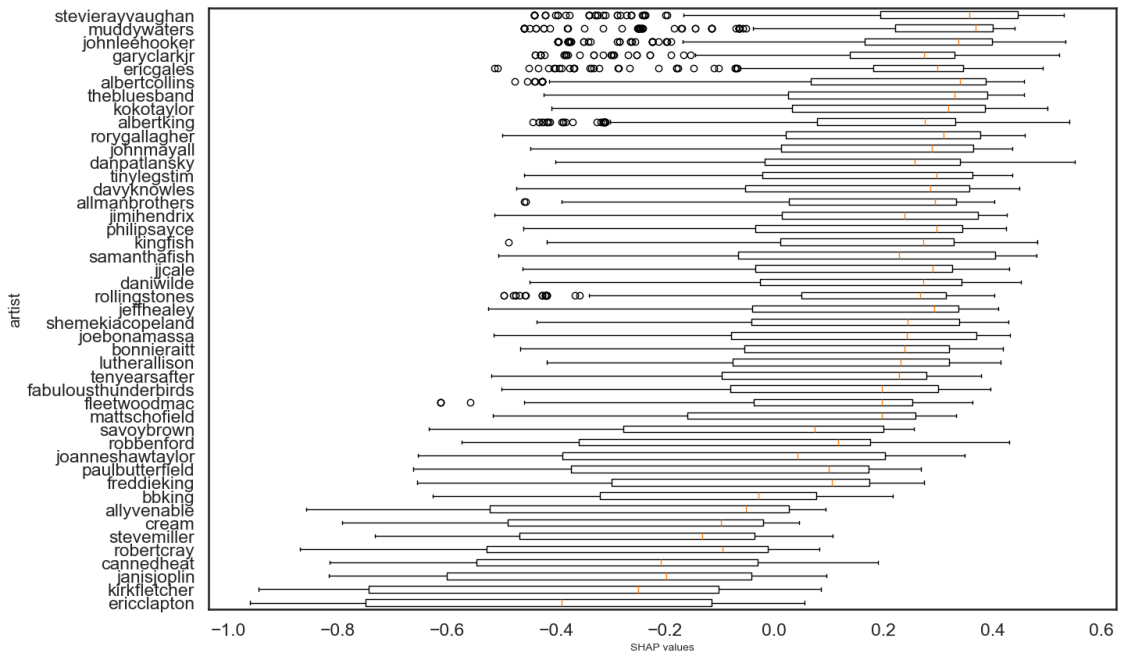
[ ]:
```
# get shap values and data
feature_values = shap_values[:, "aae_feature"].values
feature_data = shap_values[:, "aae_feature"].data

# get list of features
feature_list = df["aae_feature"].unique()

# split shap values based on feature
feature_groups = []
for a in feature_list:
    relevant_values = feature_values[feature_data == a]
    feature_groups.append(relevant_values)

# calculate the mean SHAP value for each feature
mean_shap_values = [np.mean(values) for values in feature_groups]

# sort features based on their mean SHAP values
sorted_indices = np.argsort(mean_shap_values)
sorted_feature_list = feature_list[sorted_indices]
sorted_feature_groups = [feature_groups[i] for i in sorted_indices]
```

```
# plot boxplot with sorted data
plt.boxplot(sorted_feature_groups, labels=sorted_feature_list, vert=False)
plt.xlabel("SHAP values", size=15)
plt.ylabel("feature", size=15)
plt.xticks(rotation=0)
plt.show()
```

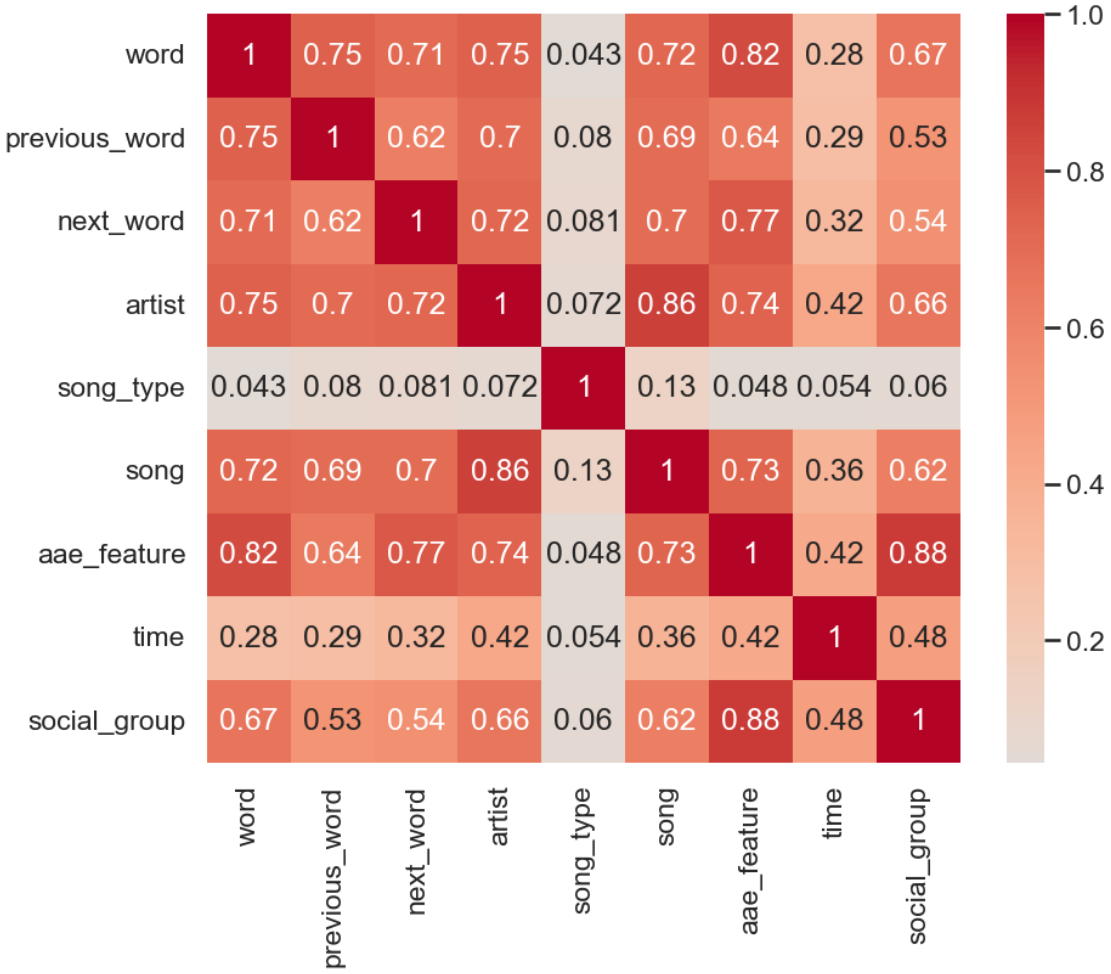


[ ]:
```
# get shap values and data
artist_values = shap_values[:, "artist"].values
artist_data = shap_values[:, "artist"].data

# get list of artists
artist_list = df["artist"].unique()

# split shap values based on artist
artist_groups = []
for a in artist_list:
    relevant_values = artist_values[artist_data == a]
    artist_groups.append(relevant_values)

# calculate the mean SHAP value for each artist
mean_shap_values = [np.mean(values) for values in artist_groups]

# sort artists based on their mean SHAP values
sorted_indices = np.argsort(mean_shap_values)
sorted_artist_list = artist_list[sorted_indices]
```

```
sorted_artist_groups = [artist_groups[i] for i in sorted_indices]

# plot boxplot with sorted data
plt.figure(figsize=(15, 10))
plt.boxplot(sorted_artist_groups, labels=sorted_artist_list, vert=False)
plt.ylabel("artist", size=15)
plt.xlabel("SHAP values", size=10)
plt.xticks(rotation=0)
plt.show()
```



[ ]:
```
# correlation matrix of shap values for each feature
corr_matrix = pd.DataFrame(shap_values.values, columns=X.columns).corr()
plt.figure(figsize=(10, 8))
sns.heatmap(corr_matrix, cmap="coolwarm", center=0.0, annot=True)
plt.show()
```


word 1 0.75 0.71 0.75 0.043 0.72 0.82 0.28 0.67
previous_word 0.75 1 0.62 0.7 0.08 0.69 0.64 0.29 0.53
next_word 0.71 0.62 1 0.72 0.081 0.7 0.77 0.32 0.54
artist 0.75 0.7 0.72 1 0.072 0.86 0.74 0.42 0.66
song_type 0.043 0.08 0.081 0.072 1 0.13 0.048 0.054 0.06
song 0.72 0.69 0.7 0.86 0.13 1 0.73 0.36 0.62
aae_feature 0.82 0.64 0.77 0.74 0.048 0.73 1 0.42 0.88
time 0.28 0.29 0.32 0.42 0.054 0.36 0.42 1 0.48
social_group 0.67 0.53 0.54 0.66 0.06 0.62 0.88 0.48 1
word
previous_word
next_word
artist
song_type
song
aae_feature
time
social_group
1.0
0.8
0.6
0.4
0.2